1. Show $\neg A \vee B, A; \cdot \vdash B$ true is derivable.

$$\dfrac{\dfrac{\neg A \vee B, A; B \vdash \neg A \vee B \text{ true}}{\dfrac{\neg A \vee B, A; B \vdash \neg A \text{ false}}{\neg A \vee B, A; B \vdash \text{contr}}} \quad \dfrac{\neg A \vee B, A; B \vdash B \text{ false}}{\neg A \vee B, A; B \vdash \neg A \vee B \text{ false}}}{\neg A \vee B, A; \cdot \vdash B \text{ true}}$$

2. Show that $\neg(\neg A \wedge \neg B); \cdot \vdash A \vee B$ true is derivable.

$$\dfrac{\dfrac{\dfrac{

Suppose $\Gamma; \Delta \vdash A$ true and $\Gamma, A; \Delta \vdash C$ true.
RTP: $\Gamma; \Delta \vdash C$ true. Effectively, $A \wedge (A \to C) \Rightarrow C$.

$$\dfrac{\Gamma; \Delta, C \vdash A \text{ true} \qquad \dfrac{\dfrac{\Gamma, A; \Delta, C \vdash C \text{ true} \qquad \Gamma, A; \Delta, C \vdash C \text{ false}}{\Gamma, A; \Delta, C \vdash \text{contr}}}{\Gamma; \Delta, C \vdash A \text{ false}}}{\dfrac{\Gamma; \Delta, C \vdash \text{contr}}{\Gamma; \Delta \vdash C \text{ true}}}$$

This derivation tree assumes weakening holds, thus $\Gamma; \Delta, C \vdash A$ true and $\Gamma, A; \Delta, C \vdash C$ true hold.

(b) In System F, consider an arbitrary type A.

(i) Give two terms $f : A \to \forall a.\, (A \to a) \to a$ and $g : (\forall a.\, (A \to a) \to a) \to A$. [3 marks]

$f : A \to \forall a.\, (A \to a) \to a$

$$\lambda x : A.\, \Lambda \alpha.\, \lambda f : (A \to \alpha).\, f\ x$$

$g : (\forall a.\, (A \to a) \to a) \to A$

$$\lambda g : (\forall a.\, (A \to a) \to a).\, g\ A\ (\lambda x : A.\, x)$$

(ii) Carefully explain what this tells you about the relationship between the types A and $\forall a.\, (A \to a) \to a$. [4 marks]

$g \circ f$ would be the identity relation, and maybe in this case we can think about f as an introduction rule and g as its elimination rule.

(c) Consider the following piece of Agda code, where `Nat` is the type of natural numbers:

```
X : (P : Nat → Set) →
    P 0 →
    ((n : Nat) → P n → P (1 + n)) →
    (k : Nat) → P k
X P base step zero = base
X P base step (suc n) = step n (X P base step n)
```

(i) Explain what the `X` function means in logical terms. [2 marks]

unsure what this is getting at

(ii) Explain what the `X` function does in terms of functional programming. [3 marks]

X is an iteration function - it considers a base value and a step function, and applies step n times to the base value based off the n supplied

2 2020 Paper 8 Question 15

(a) Recall that in constructive logic, logical negation is defined using implication and falsehood as $\neg A \triangleq A \supset \bot$.

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2020p8q15.pdf

(i) Does $A \supset \neg\neg A$? If so, give a simply-typed lambda-term corresponding to this implication.

Yes. $A \supset \neg\neg A$ is the same as $A \supset ((A \supset \bot) \supset \bot)$.
The term would be:

$$\lambda x : A.\, \lambda k : \neg A.\, k\ x$$

(ii) Does $\neg\neg A \supset A$? If so, give a simply-typed lambda-term corresponding to this implication.

No.

(iii) Does $\neg\neg\neg A \supset \neg A$? If so, give a simply-typed lambda-term corresponding to this implication.

Yes, $\neg\neg\neg A \supset \neg A$.

$$\lambda q : \neg\neg\neg A.\, \lambda x : A.\, q\ (\lambda k : \neg A.\, k\ x)$$

[5 marks]

(b) (i) Give the typing rules for Peano natural numbers and their eliminator. [2 marks]

$$\frac{}{\Gamma \vdash z : \mathbb{N}} \qquad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash s(n) : \mathbb{N}}$$

$$\frac{\Gamma \vdash e_1 : \mathbb{N} \quad \Gamma \vdash e_2 : C \quad \Gamma, x : C \vdash e_3 : C}{\Gamma \vdash \mathsf{iter}(e_1, z \to e_2, s(x) \to e_3) : C}$$

(ii) Using the rules given above, define the addition function. [3 marks]

$$\mathsf{add}\ m\ n = \lambda m : \mathbb{N}.\, \lambda n : \mathbb{N}.\, \mathsf{iter}(m,\ z \to n,\ s(x) \to s(x))$$

(iii) Let a binary tree be either a leaf `Leaf` or a node `Node(l,x,r)` where `l` and `r` are subtrees, and `x` is a natural number. Give typing rules for trees corresponding to this prose description, including an eliminator. [3 marks]

$$\frac{}{\Gamma \vdash \mathsf{Leaf} : \mathsf{BTree}} \qquad \frac{\Gamma \vdash l : \mathsf{BTree} \quad \Gamma \vdash r : \mathsf{BTree} \quad \Gamma \vdash x : \mathbb{N}}{\Gamma \vdash \mathsf{Node}(l, x, r) : \mathsf{BTree}}$$

$$\frac{\Gamma \vdash e_1 : \mathsf{BTree} \quad \Gamma \vdash e_2 : C \quad \Gamma, l : C, r : C, x : \mathbb{N} \vdash e_3 : C}{\Gamma \vdash \mathsf{iter}(e_1,\ \mathsf{Leaf} \to e_2,\ \mathsf{Node}(l, x, r) \to e_3) : C}$$

(iv) Using the rules given above, define a function `size` which takes a binary tree and returns the total number of nodes in the tree. [5 marks]

$$\mathsf{size} = \lambda x : \mathsf{BTree}.\, \mathsf{iter}(x,\ \mathsf{Leaf} \to z,\ \mathsf{Node}(l, x, r) \to s(\mathsf{add}\ l\ r))$$

(c) The `zip` function takes two lists, and returns a list of pairs of the elements as output. Suppose we see the following Agda type declaration for `zip`:

$$zip : \forall\{A\ B : Set\} \to \{n : Nat\} \to \mathsf{Vec}\, A\, n \to \mathsf{Vec}\, B\, n \to \mathsf{Vec}\, (A \times B)\, n$$

Explain what this means in terms of how to call the function, and what properties the result has. [2 marks]

When we call the zip function, we need to minimally supply 2 lists of types A and B, each of length n. We can optionally supply this value n, which must match the length of the list supplied, else this will not typecheck. We can also optionally supply the types of the elements of both list. The output will be guarenteed to have a length n, and its elements will be of a type pair of A and B.

3 2019 Paper 8 Question 13

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2019p8q13.pdf

Recall the three judgements for classical propositional logic:

(a) $\Gamma; \Delta \vdash e : A \ \mathsf{true}$ – e is a proof of type A

(b) $\Gamma; \Delta \vdash k : A \ \mathsf{false}$ – k is a refutation of type A

(c) $\Gamma; \Delta \vdash \langle e \mid_A k \rangle \ \mathsf{contr}$ – $\langle e \mid_A k \rangle$ is a contradiction at type A

Here, Γ contains the true assumptions, and Δ are the false assumptions. In this question, we will extend classical propositional logic with support for the implication or function type operator $A \to B$.

(a) Give a proof term and inference rule for a proof of type $A \to B$. [4 marks]

$$\frac{\Gamma, x : A; \Delta \vdash e : B \ \mathsf{true}}{\Gamma; \Delta \vdash \lambda x : A.\, e : A \to B \ \mathsf{true}}$$

(b) Give a proof term and inference rule for a refutation of type $A \to B$.

[*Hint*: how is implication encoded in classical logic?] [4 marks]

We note that $A \to B$ is encoded as $\neg A \vee B$ in classical logic. Writing this out, we get:

$$\frac{\dfrac{\Gamma; \Delta \vdash A \ \mathsf{true}}{\Gamma; \Delta \vdash \neg A \ \mathsf{false}} \qquad \Gamma; \Delta \vdash k : B \ \mathsf{true}}{\Gamma; \Delta \vdash [\neg e, k] : \neg A \vee B \ \mathsf{false}}$$

Hence the rule is:

$$\frac{\Gamma; \Delta \vdash e : A \ \mathsf{true} \quad \Gamma; \Delta \vdash k : B \ \mathsf{false}}{\Gamma; \Delta \vdash [\neg e, k] : A \to B \ \mathsf{false}}$$

I suppose if we follow the notation that $[.,.]$ is a sum of continuations, we might intepret $\neg e$ here as a value that has somehow been pushed onto the continuation stack, and hence could be sued later down the road.

(c) Give a reduction rule for contradiction configurations of the form $\langle e \mid_{A \to B} k \rangle$. [4 marks]

$\langle e \mid_{A\to B} k\rangle$ must be of the form $\langle \lambda x : A.\, e' \mid_{A\to B} [e, k]\rangle$ after reduction, so

$$\langle \lambda x : A.\, e' \mid_{A\to B} [e, k]\rangle \rightsquigarrow \langle [e/x]e' \mid_B k\rangle$$

(d) (i) State the preservation theorem for classical logic. [2 marks]

If $c = \langle e \mid_A k\rangle$, $\cdot;\cdot \vdash c$ contr $\wedge\ c \rightsquigarrow c'$ then $\cdot;\cdot \vdash c'$ contr.

(ii) Give the proof of preservation for the case of the new rule defined above. You may assume that weakening, exchange and substitution all hold. [6 marks]

Assume that $\cdot;\cdot \vdash \langle \lambda x : A.\, e' \mid_{A\to B} [e, k]\rangle$ contr
and $\langle \lambda x : A.\, e' \mid_{A\to B} [e, k]\rangle \rightsquigarrow \langle [e/x]e' \mid_B k\rangle$.
RTP: $\cdot;\cdot \vdash \langle [e/x]e' \mid_B k\rangle$ contr.
Assume weakening (of contexts), exchange (of contexts) and substitution (of terms of appropriate types) are true.
By inversion, $\cdot;\cdot \vdash \lambda x : A.\, e' : A \to B$ true (1)
and $\cdot;\cdot \vdash [e, k] : A \to B$ false. (2)
By inversion of (1), $x : A;\cdot \vdash e' : B$ true. (3)
By inversion of (2), $\cdot;\cdot \vdash e : A$ true (4)
and $\cdot;\cdot \vdash k : B$ false. (5)
By substitution, (3) and (4), $\cdot;\cdot \vdash [e/x]e' : B$ true. (6)
So by (5) and (6), $\cdot;\cdot \vdash \langle [e/x]e' \mid_B k\rangle$ contr.

4 2014 Paper 9 Question 13

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2014p9q13.pdf

(a) Give an account of the Polymorphic Lambda Calculus (PLC). You should define the PLC types, expressions and typing environments, give the PLC typing rules, and define the relations of *beta-reduction* and *beta-conversion*. Explain why beta-conversion is a decidable relation for expressions that are typeable. [10 marks]

Polymorphic lambda calculus:

$$\begin{aligned}
\text{Types } A, B &::= \alpha \mid A \to B \mid \forall\alpha.\, A \\
\text{Terms } e &::= x \mid \lambda x : A.\, e \mid e\ e \mid \Lambda\alpha.\, e \mid e\ A \\
\text{Term contexts } \Gamma &::= \cdot \mid \Gamma, x : A \\
\text{Type contexts } \Theta &::= \cdot \mid \Theta, \alpha \\
\text{Values } v &::= x \mid \lambda x : A.e \mid \Lambda\alpha.e
\end{aligned}$$

$$\frac{\alpha \in \Theta}{\Theta \vdash \alpha \text{ type}} \qquad \frac{\Theta \vdash A \text{ type} \quad \Theta \vdash B \text{ type}}{\Theta \vdash A \to B \text{ type}} \qquad \frac{\Theta, \alpha \vdash A \text{ type}}{\Theta \vdash \forall\alpha.\, A \text{ type}}$$

$$\frac{}{\Theta \vdash \cdot \text{ ctx}} \qquad \frac{\Theta \vdash \Gamma \text{ ctx} \quad \Theta \vdash A \text{ type}}{\Theta \vdash \Gamma, x : A \text{ ctx}}$$

$$\frac{x : A \in \Gamma}{\Theta;\Gamma \vdash x : A} \qquad \frac{\Theta;\Gamma, x : A \vdash e : B}{\Theta;\Gamma \vdash \lambda x : A.\, e : A \to B}$$

$$\frac{\Theta;\Gamma \vdash e_1 : A \to B \quad \Theta;\Gamma \vdash e_2 : A}{\Theta;\Gamma \vdash e_1\ e_2 : B} \qquad \frac{\Theta, \alpha;\Gamma \vdash e : A}{\Theta;\Gamma \vdash \Lambda\alpha.\, e : \forall\alpha.\, A}$$

$$\frac{\Theta, \alpha; \Gamma \vdash e : B \quad \Theta \vdash A \text{ type}}{\Theta; \Gamma \vdash e\ A : [A/\alpha]B}$$

We might then define the beta-reduction with the following rules:

$$\frac{e_1 \to e_1'}{e_1 e_2 \to e_1' e_2} \qquad \frac{e \to e'}{v\ e \to v\ e'}$$

$$\frac{}{(\lambda x : A.e)v \to [v/x]e}$$

$$\frac{e \to e'}{eA \to e'A} \qquad \frac{}{(\Lambda\alpha.e)A \to [A/\alpha]e}$$

Here, we further define beta-conversion as: two terms are beta-equivalent if one can be derived from the other in zero or more steps of beta reduction, beta expansion or alpha conversion.
For PLC, we know we have proven type safety and termination. Given this and the fact that beta-reduction is confluent, we know that equivalent will eventually reduce to the same 'normal' form, and thus equivalence is decideable.

(b) Find, with justification, a PLC type τ for which the following typings are both provable:

$$\{\} \vdash \Lambda\alpha, \beta\,(\lambda x : \alpha\,(\lambda y : \beta\,(\Lambda\gamma\,(\lambda z : \tau\,(z\,x\,y)))))$$
$$: \forall\alpha, \beta\,(\alpha \to (\beta \to \forall\gamma\,(\tau \to \gamma)))$$

$$\{\} \vdash \Lambda\alpha, \beta\,(\lambda z : \forall\gamma\,(\tau \to \gamma)\,(z\,\alpha\,(\lambda x : \alpha\,(\lambda y : \beta\,(x)))))$$
$$: \forall\alpha, \beta\,((\forall\gamma\,(\tau \to \gamma)) \to \alpha)$$

[5 marks]

In the first typing, we note thast z which is of type τ takes in $x : \alpha$ and $y : \beta$. Its type signature suggests that the output of z will be of type γ.
In the second typing, we again see something similar, so we note that τ will be of type $\alpha \to \beta \to \gamma$.

(c) Give infinitely many different closed PLC expressions in beta-normal form of type $\forall\alpha\,((\alpha \to \alpha) \to (\alpha \to \alpha))$. [2 marks]

$\forall\alpha.\,(\alpha \to \alpha) \to (\alpha \to \alpha)$

$$\Lambda\alpha.\,\lambda f : (\alpha \to \alpha).\,\lambda x : \alpha.\,f^n\,x$$

(d) Use your answer to part (a) to show that there is no closed PLC expression Y of type $\forall\alpha\,((\alpha \to \alpha) \to \alpha)$ for which the beta-reduction $Y\,\alpha\,f \to f(Y\,\alpha\,f)$ holds. [3 marks]

$Y : \forall\alpha.\,(\alpha \to \alpha) \to \alpha$
We need to prove there is no Y satisfying $Y\,\alpha\,f \to f(Y\,\alpha\,f)$, where f is of type $\alpha \to \alpha$.
We note that if the beta reduction holds, we would have an infinitely reducing chain looking like $Y\,\alpha\,f \to f(Y\,\alpha\,f) \to f(f(Y\,\alpha\,f)) \to \cdots$, which contradicts the claim that all well-typed terms in PLC terminate.

5 2011 Paper 8 Question 14

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2011p8q14.pdf

Terms in the polymorphic lambda calculus (PLC) are given by the grammar

$$M ::= x \mid \lambda x : \tau\,(M) \mid M\,M \mid \Lambda\alpha\,(M) \mid M\,\tau$$

where τ is a type, α a type variable and x a variable.

(a) Give the rules for the type system in PLC. [2 marks]

$$\frac{x : A \in \Gamma}{\Theta;\Gamma \vdash x : A} \qquad \frac{\Theta;\Gamma, x : A \vdash e : B}{\Theta;\Gamma \vdash \lambda x : A.\, e : A \to B}$$

$$\frac{\Theta;\Gamma \vdash e_1 : A \to B \quad \Theta;\Gamma \vdash e_2 : A}{\Theta;\Gamma \vdash e_1\ e_2 : B} \qquad \frac{\Theta,\alpha;\Gamma \vdash e : A}{\Theta;\Gamma \vdash \Lambda\alpha.\, e : \forall\alpha.\, A}$$

$$\frac{\Theta,\alpha;\Gamma \vdash e : B \quad \Theta \vdash A\ \mathsf{type}}{\Theta;\Gamma \vdash e\ A : [A/\alpha]B}$$

(b) Give the rules for the relation, $\to_\beta$, of beta-reduction in PLC. Explain what it means for a term to be beta-normal. [3 marks]

$$\frac{M_1 \to M_1'}{M_1\,M_2 \to M_1'\,M_2} \qquad \frac{M \to M'}{v\,M \to v\,M'}$$

$$(\lambda x : \tau.\, M)\,M' \to [M'/x]M$$

$$\frac{M \to M'}{M\,\tau \to M'\,\tau} \qquad (\Lambda\alpha.\, M)\,\tau \to [\tau/\alpha]M$$

where

$$v ::= x \mid \lambda x : \tau.\, M \mid \Lambda\alpha.\, M$$

Beta normal if it is a value.
(Not sure if we normally include this, but seems like we need this for c)

$$\frac{M \to M'}{\lambda x : A.M \to \lambda x : A.M'}$$

(c) Terms in *head-normal form*, H, can be described in PLC by the grammar

$$A ::= x \mid A\,H \mid A\,\tau$$
$$H ::= A \mid \lambda x : \tau\,(H) \mid \Lambda\alpha\,(H)$$

Arguing by induction on the structure of terms, or otherwise, prove that every term in PLC that is both typable and beta-normal is of head-normal form. [8 marks]

$$A ::= x \mid A\,H \mid A\,\tau$$
$$H ::= A \mid \lambda x : \tau.\, H \mid \Lambda\alpha\,(H)$$

RTP: If $\Theta;\Gamma \vdash e : A \wedge e \not\to$, then e is in head normal form.
We proceed by structural induction on

$$M ::= x \mid \lambda x : \tau.\, M \mid M\,M \mid \Lambda\alpha.\, M \mid M\,\tau$$

Consider

$$\Phi(e) \triangleq \text{if } \Gamma \vdash e : T \wedge e \not\to \text{ then } e \text{ is in head normal form}$$

Case 1: x. Assume $\Gamma \vdash x : T \wedge x \not\to$. Trivially, e is in head normal form.
Case 2: $\lambda x : \tau . M$.
By inversion, we know M is typeable. We further note that because this is in beta normal form, M is beta-normal as well. Hence, M is head normal, and thus $\lambda x : \tau . M$ is also head-normal
Case 3: $\Lambda \alpha . M$.
By inversion, we know M is typeable. We further note that because this is in beta normal form, M is beta-normal as well. Hence, M is head normal, and thus $\Lambda \alpha . M$ is also head-normal
Case 4: $M_1\, M_2$. Assume $\Phi(M_1) \wedge \Phi(M_2)$. By a similar argument of typability and beta-normal of $M_1 M_2$, we have that M_1 and M_2 are both head-normal. Looking at M_1, it cannot be of form $\lambda x : \tau . H$ else it would be able to reduce, hence is not beta normal, and also cannot be of form $\Lambda \alpha . H$, as it would not type check. This thus must be of the form A, and hence we can see that the term is of head-nromal form.
Case 5: $M\tau$. Assume $\Phi(M)$). By a similar argument of typability and beta-normal of $M\tau$, we have that M is head-normal. Looking at M, it cannot be of form $\lambda x : \tau . H$ else it would not type check. It also cannot be of form $\Lambda \alpha . H$, as it could reduce further and not be beta normal.. This thus must be of the form A, and hence we can see that the term is of head-normal form.

(d) Natural numbers can be encoded into PLC using the type *nat* defined as

$$nat \stackrel{\text{def}}{=} \forall \alpha \, (\alpha \to (\alpha \to \alpha) \to \alpha)$$

The encoding uses the Church-numerals defined as

$$\overline{n} \stackrel{\text{def}}{=} \Lambda \alpha \, (\lambda x : \alpha \, (\lambda y : \alpha \to \alpha \, (\underbrace{y \, (y \, (y \ldots (y \, x) \ldots))}_{n \text{ occurrences}})))$$

Prove that the Church-numerals are the only closed, beta-normal terms of type *nat*.

Hint: Use the result proved in part (c) and do a case analysis over the form of terms in head-normal form. You may assume without proof that if $\Gamma \vdash M : \tau$ is provable in the PLC type system, then the free variables of the term M are contained in the domain of the typing environment Γ. [7 marks]

Type *nat* defined as

$$nat \triangleq \forall \alpha \, (\alpha \to (\alpha \to \alpha) \to \alpha)$$

Church numerals as

$$\overline{n} \triangleq \Lambda \alpha . \, \lambda z : \alpha . \, \lambda s : \alpha \to \alpha . \, s^n \, z$$

RTP: these are the only closed beta normal terms.
We note in (c) we proved that if typable and beta normal, then head normal. So we want to prove that if $\cdot \vdash e : \forall \alpha \, (\alpha \to (\alpha \to \alpha) \to \alpha)$ and e is closed and beta normal, then e is of the form $\overline{n}$.
Suppose we have another closed beta normal term of type *nat* that is not a Church numeral. By (c), it is of head normal form. Then, e is of the form A, $\lambda x : \tau \, (H)$ or $\Lambda \alpha . \, H$. Repeating this argument, It can only be of the form

$$\Lambda \alpha . \, \lambda x : \alpha . \, \lambda y : \alpha \to \alpha . \, H$$

The last H must be of some form A; given x and y and output α, it can only be some number of applications. This, this term must be of the form

$$\Lambda \alpha . \, \lambda x : \alpha . \, \lambda y : \alpha \to \alpha . \, y^n x$$